

how to recognize a sliding-window problem

say “window” when you see any of these cues:

1. contiguous “**substring/subarray**” + optimize (longest/shortest/max/min/count).
2. “**at most k** replacements/zeros/distinct chars” → grow + shrink.
3. “**fixed length k**” → move a window of size k.
4. “**contains all chars of P** / anagrams / permutation” → need/counter + shrink to minimum.
5. **sum/product threshold** with **positive numbers** → expand until broken, then shrink.
6. “**exactly k** distinct/odds/zeros” → compute `atMost(k) - atMost(k-1)`.

if negatives are involved with sums, two-pointers often fail → think prefix sums or monotonic deque (flagged below).

A) fixed-size window (size = k)

- **use when:** window size fixed by problem.
- **state:** running sum / count / freq updated by add-right, remove-left.
- **examples:** max sum/avg of size k, max vowels in window of size k, “max in sliding window”.

template (sum/avg/count)

```
int best = INT_MIN, sum = 0;
for (int r = 0; r < n; ++r) {
    sum += a[r];
    if (r >= k) sum -= a[r - k];
    if (r >= k - 1) best = max(best, sum);
}
```

sliding window maximum (monotonic deque)

- **state:** deque of indices with values in decreasing order; front = max.

```
deque<int> dq;
vector<int> ans;
for (int r = 0; r < n; ++r) {
    while (!dq.empty() && a[dq.back()] <= a[r]) dq.pop_back();
    dq.push_back(r);
    if (dq.front() <= r - k) dq.pop_front();
    if (r >= k - 1) ans.push_back(a[dq.front()]);
}
```

B) variable-size “at most K something”

- **use when:** “at most k replacements / at most k distinct / at most k zeros”.
- **state:** freq/counters; **valid if** constraint satisfied.
- **mechanic:** expand `r`; while invalid → shrink `l`; update answer.

general template

```
int l = 0, ans = 0;
unordered_map<int,int> cnt;
int bad = 0; // measure of violation (e.g., distinct > K or zeros > K)

for (int r = 0; r < n; ++r) {
    // add a[r]; update cnt/bad
    while (/* invalid: bad > K */) {
        // remove a[l++]; update cnt/bad
        ++l;
    }
    ans = max(ans, r - l + 1);
}
return ans;
```

common specializations

1) longest substring with at most K distinct chars

```
int l = 0, ans = 0, distinct = 0;
array<int,256> f{}; // fixed alphabet faster than unordered_map
for (int r = 0; r < s.size(); ++r) {
    if (f[(unsigned char)s[r]]++ == 0) ++distinct;
    while (distinct > K) {
        if (--f[(unsigned char)s[l++]] == 0) --distinct;
    }
    ans = max(ans, r - l + 1);
}
```

2) longest repeating char replacement (your earlier problem)

invariant: (window_len - maxFreq) <= k

(trick: keep maxFreq monotone increasing; do not recompute on shrink)

```
int l = 0, ans = 0, maxFreq = 0;
array<int,26> f{};
for (int r = 0; r < s.size(); ++r) {
    int idx = s[r] - 'A';
    maxFreq = max(maxFreq, ++f[idx]);
    while ((r - l + 1) - maxFreq > k) {
        --f[s[l++] - 'A'];
    }
    ans = max(ans, r - l + 1);
}
```

3) max consecutive ones III (flip ≤ k zeros)

maintain zeros count; invalid if zeros > k

C) “exactly K” via at-most trick

- **use when:** exactly K distinct/odds/zeros.
- **idea:** $\text{countExactly}(K) = \text{countAtMost}(K) - \text{countAtMost}(K-1)$.

- **for counting subarrays**, not for max length.

countAtMost template (returns number of valid subarrays)

```
long long atMostKDistinct(const vector<int>& a, int K) {
    unordered_map<int,int> f;
    int l = 0; long long res = 0; int distinct = 0;
    for (int r = 0; r < a.size(); ++r) {
        if (f[a[r]]++ == 0) ++distinct;
        while (distinct > K) {
            if (--f[a[l++]] == 0) --distinct;
        }
        res += (r - l + 1); // all subarrays ending at r, starting l..r valid
    }
    return res;
}
// countExactlyK = atMost(K) - atMost(K-1)
```

common uses: subarrays with exactly k distinct ints; subarrays with exactly k odd numbers (treat odd as 1/0 and “distinct” replaced by a sum-based atMost as needed).

D) sum/product thresholds with positives

- **use when:** “smallest subarray with sum $\geq$ target”, “count subarrays with product $< k$ ”.
- **requirement:** **all numbers nonnegative** (positives ideally). otherwise, sliding window is unreliable.

1) min length with sum $\geq$ target (positives)

```
int l = 0, sum = 0, best = INT_MAX;
for (int r = 0; r < n; ++r) {
    sum += a[r];
    while (sum >= target) {
        best = min(best, r - l + 1);
        sum -= a[l++];
    }
}
return best == INT_MAX ? 0 : best;
```

2) count subarrays with product $< k$ (positives)

```
long long res = 0;
long double prod = 1; // or double; careful with overflow
int l = 0;
for (int r = 0; r < n; ++r) {
    prod *= a[r];
    while (l <= r && prod >= k) prod /= a[l++];
    res += (r - l + 1);
}
return res;
```

E) covering/requirement windows (need vs have)

- **use when:** “min window substring”, “find all anagrams”, “permutation in string”.
- **state:** need map with required counts; have/formed for matched characters.
- **mechanic:** expand until all required satisfied, then **shrink to minimum**.

min window substring (classic)

```
string minWindow(string s, string t) {
    array<int,128> need{}; int needKinds = 0;
    for (char c : t) if (need[(int)c]++ == 0) ++needKinds;

    array<int,128> have{};
    int formed = 0, l = 0;
    int bestL = -1, bestLen = INT_MAX;

    for (int r = 0; r < s.size(); ++r) {
        int c = s[r];
        if (++have[c] == need[c] && need[c] > 0) ++formed;

        while (formed == needKinds) {
            if (r - l + 1 < bestLen) bestLen = r - l + 1, bestL = l;
            int d = s[l++];
            if (need[d] > 0 && --have[d] < need[d]) --formed;
        }
    }
    return bestL == -1 ? "" : s.substr(bestL, bestLen);
}
```

find all anagrams / permutation in string

Use fixed-size window equal to $|p|$; compare counts (or maintain a match counter).

F) counting windows vs maximizing length

- **maximize/minimize length** $\rightarrow$ `ans = max(ans, r-l+1)` OR `best = min(best, r-l+1)` after valid.
- **count subarrays** where each end index contributes multiple starts $\rightarrow$ add `ans += (r - l + 1)` whenever window valid (see atMost trick and product<k).

G) when sliding window is not enough

- **negatives allowed** in sum constraints (e.g., shortest subarray with $\text{sum} \geq k$ with negatives) $\rightarrow$ use **prefix sums + monotonic deque** (not two pointers).
- “number of subarrays with $\text{sum} = k$ ” $\rightarrow$ **prefix sum + hashmap**.
- “longest substring without repeating chars” is sliding window, but “even counts / palindrome permutations” may need **bitmask**, not window.
- if order isn’t contiguous $\rightarrow$ not a window (use sets / sorting / two-sum style).

H) circular arrays

- duplicate array to simulate wrap (i in $[0, 2n)$), but **bound window length $\leq n$** . or use mod arithmetic carefully.
-

quick decision tree (mental flow)

1. **contiguous?** if no $\Rightarrow$ not sliding window.
 2. **fixed K length?** $\rightarrow$ fixed-size window (sum/freq/deque).
 3. **need to “cover” a pattern?** $\rightarrow$ need/have + shrink to min.
 4. **constraint is “at most K X” (distinct/zeros/replacements)?** $\rightarrow$ variable window + while invalid shrink.
 5. **want exactly K?** $\rightarrow \text{atMost}(K) - \text{atMost}(K-1)$.
 6. **sum/product threshold and all values ≥ 0 ?** $\rightarrow$ variable window.
 7. **negatives appear in sum constraints?** $\rightarrow$ switch to prefix sums/deque (not window).
-

common invariants & tricks (the “why”)

- **validity check monotonicity:** expanding never decreases window length; shrinking restores validity. this makes the two-pointer linear.
- **`ans += r-l+1`:** when counting subarrays for at-most constraints; every start between $l \dots r$ gives a valid subarray ending at r .
- **maxFreq trick (char replacement):** keep `maxFreq` as the **max seen so far**; it can be stale when shrinking but correctness holds (you might shrink more; you won't miss a global optimum).
- **maps vs arrays:** string alphabets $\Rightarrow$ `array<int, 26/52/128/256>` beats `hashmap` on speed.
- **off-by-one:** in fixed-size windows, remove `a[r-k]` **after** `r >= k`. in variable windows, be careful with index used when removing

12-problem drill (covers all patterns)

1. Fixed-size sum/avg: Max avg subarray I (size k).
2. Fixed-size frequency: Max vowels in substring of size k.
3. Sliding window maximum (deque).
4. At most K distinct: Longest substring with at most K distinct.
5. K = 2 specialization: Fruit into baskets.
6. Replacement: Longest repeating character replacement.
7. Zeros flips: Max consecutive ones III.
8. Sum threshold (positives): Min subarray length $\geq$ target.
9. Product threshold: Subarray product $< k$ (count).
10. Covering (min): Minimum window substring.
11. Anagrams: Find all anagrams in a string (fixed-size window + counts).
12. Exactly K distinct (counting): Subarrays with K different integers.

Fixed-size sum/avg:

◆ Pattern: Fixed-size Sliding Window

- **When?** → If the problem explicitly fixes a window size k and asks about **sum/avg/max/min/count** in such subarrays.
- **Trick to remember:**
👉 “Window size is fixed → just slide by dropping left and adding right.”
- **Time complexity:** $O(n)$, instead of brute-force $O(n*k)$.

✓ Variation 1: Max Average Subarray I

Q: Given an array `nums` and integer k , return max average of any subarray of length k .

Trick: Maintain a window sum of size k .

- Add `nums[right]`, remove `nums[left]` when size $> k$.
- Keep max sum.

```
int findMaxAverage(vector<int>& nums, int k) {
    long sum = 0, maxSum = LLONG_MIN;
    int n = nums.size();
    for(int i=0; i<n; i++){
        sum += nums[i];
```

```

        if(i >= k) sum -= nums[i-k];
        if(i >= k-1) maxSum = max(maxSum, sum);
    }
    return (double)maxSum / k;
}

```

✓ Variation 2: Max Sum Subarray of Size k

Q: Given array and k, return max sum of any subarray of size k.

Trick: Same as above, but no division.

```

int maxSumSubarray(vector<int>& nums, int k) {
    int sum = 0, maxSum = INT_MIN;
    for(int i=0; i<nums.size(); i++){
        sum += nums[i];
        if(i >= k) sum -= nums[i-k];
        if(i >= k-1) maxSum = max(maxSum, sum);
    }
    return maxSum;
}

```

✓ Variation 3: Min Sum Subarray of Size k

Q: Return minimum sum of any window of size k.

Trick: Just track min instead of max.

```

int minSumSubarray(vector<int>& nums, int k) {
    int sum = 0, minSum = INT_MAX;
    for(int i=0; i<nums.size(); i++){
        sum += nums[i];
        if(i >= k) sum -= nums[i-k];
        if(i >= k-1) minSum = min(minSum, sum);
    }
    return minSum;
}

```

✓ Variation 4: First Negative Number in Every Window of Size k

Q: Return array of first negative number in each window of size k. If no negative → 0.

Trick: Keep a deque of negative numbers' indices.

```

vector<int> firstNegatives(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> ans;
    for(int i=0; i<nums.size(); i++){

```

```

        if(nums[i] < 0) dq.push_back(i);
        if(i >= k-1){
            if(!dq.empty() && dq.front() <= i-k) dq.pop_front();
            ans.push_back(dq.empty()? 0 : nums[dq.front()]);
        }
    }
    return ans;
}

```

✓ Variation 5: Count Distinct Elements in Every Window

Q: Given array & k , return count of distinct numbers in every window of size k .

Trick: Use hashmap to track frequency.

```

vector<int> distinctInWindow(vector<int>& nums, int k) {
    unordered_map<int,int> mp;
    vector<int> ans;
    for(int i=0;i<nums.size();i++){
        mp[nums[i]]++;
        if(i >= k){
            if(--mp[nums[i-k]] == 0) mp.erase(nums[i-k]);
        }
        if(i >= k-1) ans.push_back(mp.size());
    }
    return ans;
}

```

✓ Variation 6: Maximum in Every Window of Size k (Sliding Window Maximum)

Q: Return max in each window.

Trick: Use deque (monotonic decreasing).

```

vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> ans;
    for(int i=0;i<nums.size();i++){
        while(!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();
        dq.push_back(i);
        if(dq.front() <= i-k) dq.pop_front();
        if(i >= k-1) ans.push_back(nums[dq.front()]);
    }
    return ans;
}

```

✓ Variation 7: Minimum in Every Window

Trick: Same as max, but maintain monotonic **increasing deque**.



How to Recognize Fixed-Size Window?

- Problem **gives k explicitly**.
- You're asked about **sum, avg, max, min, distinct count, negative, etc.** in each subarray of length k.
- **Template:**
- ```
for i in [0..n):
```
- ```
    add nums[i]
```
- ```
 if i >= k: remove nums[i-k]
```
- ```
    if i >= k-1: process answer
```

Fixed-size frequency:



Core Idea (Trick to Remember)

- You are **not summing values**, but instead **counting a property** inside a window of size k.
- Condition is usually something like *vowels, consonants, 1s in binary array, distinct elements, special chars, primes...*
- Maintain a **running frequency count** while sliding.
- Always update when a **new char enters** and an **old char leaves** the window.

👉 **Trick:**

If you see **window size = k fixed** + condition on characters/numbers → It's **Fixed-size Frequency** pattern.



Variations & Questions

1. Max Vowels in Substring of Size k (LC 1456)

Q: Find the maximum number of vowels in any substring of length k.

Trick: Count vowels as they enter/exit.

```
int maxVowels(string s, int k) {  
    unordered_set<char> vowels = {'a','e','i','o','u'};
```

```

int cnt = 0, ans = 0;
for(int i=0;i<s.size();i++){
    if(vowels.count(s[i])) cnt++;
    if(i>=k && vowels.count(s[i-k])) cnt--;
    ans = max(ans,cnt);
}
return ans;
}

```

2. Max Consonants in Substring of Size k

Q: Find maximum consonants in any substring of length k.

Trick: Same as vowels but reverse condition.

```

int maxConsonants(string s, int k) {
    unordered_set<char> vowels = {'a','e','i','o','u'};
    int cnt=0, ans=0;
    for(int i=0;i<s.size();i++){
        if(!vowels.count(s[i])) cnt++;
        if(i>=k && !vowels.count(s[i-k])) cnt--;
        ans = max(ans,cnt);
    }
    return ans;
}

```

3. Max 1s in Subarray of Size k

Q: Given binary array, find max number of 1s in any subarray of size k.

Trick: Count arr[i]==1.

```

int maxOnes(vector<int>& nums, int k) {
    int cnt=0, ans=0;
    for(int i=0;i<nums.size();i++){
        if(nums[i]==1) cnt++;
        if(i>=k && nums[i-k]==1) cnt--;
        ans=max(ans,cnt);
    }
    return ans;
}

```

4. Max Distinct Chars in Substring of Size k

Q: Find max number of distinct characters in substring of size k.

Trick: Use hashmap freq.

```

int maxDistinct(string s, int k) {
    unordered_map<char,int> freq;
    int ans=0;
    for(int i=0;i<s.size();i++){
        freq[s[i]]++;
    }
}

```

```

        if(i>=k){
            if(--freq[s[i-k]]==0) freq.erase(s[i-k]);
        }
        if(i>=k-1) ans = max(ans, (int)freq.size());
    }
    return ans;
}

```

5. Max Even Numbers in Subarray of Size k

Q: Given nums, find max count of even numbers in subarray of length k.

Trick: Count evens.

```

int maxEvens(vector<int>& nums, int k) {
    int cnt=0, ans=0;
    for(int i=0;i<nums.size();i++){
        if(nums[i]%2==0) cnt++;
        if(i>=k && nums[i-k]%2==0) cnt--;
        ans=max(ans, cnt);
    }
    return ans;
}

```

6. Max Prime Numbers in Subarray of Size k

Q: Count how many primes appear in subarray of size k, return max.

Trick: Precompute isPrime.

```

bool isPrime(int x){
    if(x<2) return false;
    for(int i=2;i*i<=x;i++) if(x%i==0) return false;
    return true;
}
int maxPrimes(vector<int>& nums, int k) {
    int cnt=0, ans=0;
    for(int i=0;i<nums.size();i++){
        if(isPrime(nums[i])) cnt++;
        if(i>=k && isPrime(nums[i-k])) cnt--;
        ans=max(ans, cnt);
    }
    return ans;
}

```

◆ Patterns You Can Derive

- **Chars:** vowels, consonants, uppercase, lowercase, digits.
- **Numbers:** evens, odds, primes, negatives, positives, multiples of k.

- **Special properties:** distinct count, freq of most occurring char, palindromic chars (if asked).
-

◆ Recognition Trick (Summary)

👉 Whenever you see:

- Substring/subarray of **fixed length k**
- Count **how many elements satisfy a property**
- Asked for **max/min count**

💡 Immediately think: **Fixed-size Frequency Sliding Window**.

◆ Core Idea (Trick to Remember)

- You are **not summing values**, but instead **counting a property** inside a window of size k.
- Condition is usually something like *vowels, consonants, 1s in binary array, distinct elements, special chars, primes....*
- Maintain a **running frequency count** while sliding.
- Always update when **a new char enters** and **an old char leaves** the window.

👉 **Trick:**

If you see **window size = k fixed** + condition on characters/numbers → It's **Fixed-size Frequency** pattern.

◆ Variations & Questions

1. Max Vowels in Substring of Size k (LC 1456)

Q: Find the maximum number of vowels in any substring of length k.

Trick: Count vowels as they enter/exit.

```
int maxVowels(string s, int k) {
```

```

unordered_set<char> vowels = {'a','e','i','o','u'};
int cnt = 0, ans = 0;
for(int i=0;i<s.size();i++){
    if(vowels.count(s[i])) cnt++;
    if(i>=k && vowels.count(s[i-k])) cnt--;
    ans = max(ans,cnt);
}
return ans;
}

```

2. Max Consonants in Substring of Size k

Q: Find maximum consonants in any substring of length k.

Trick: Same as vowels but reverse condition.

```

int maxConsonants(string s, int k) {
    unordered_set<char> vowels = {'a','e','i','o','u'};
    int cnt=0, ans=0;
    for(int i=0;i<s.size();i++){
        if(!vowels.count(s[i])) cnt++;
        if(i>=k && !vowels.count(s[i-k])) cnt--;
        ans = max(ans,cnt);
    }
    return ans;
}

```

3. Max 1s in Subarray of Size k

Q: Given binary array, find max number of 1s in any subarray of size k.

Trick: Count `arr[i]==1`.

```

int maxOnes(vector<int>& nums, int k) {
    int cnt=0, ans=0;
    for(int i=0;i<nums.size();i++){
        if(nums[i]==1) cnt++;
        if(i>=k && nums[i-k]==1) cnt--;
        ans=max(ans,cnt);
    }
    return ans;
}

```

4. Max Distinct Chars in Substring of Size k

Q: Find max number of distinct characters in substring of size k.

Trick: Use hashmap freq.

```

int maxDistinct(string s, int k) {
    unordered_map<char,int> freq;
    int ans=0;
    for(int i=0;i<s.size();i++){

```

```

        freq[s[i]]++;
        if(i>=k){
            if(--freq[s[i-k]]==0) freq.erase(s[i-k]);
        }
        if(i>=k-1) ans = max(ans, (int)freq.size());
    }
    return ans;
}

```

5. Max Even Numbers in Subarray of Size k

Q: Given nums, find max count of even numbers in subarray of length k.

Trick: Count evens.

```

int maxEvens(vector<int>& nums, int k) {
    int cnt=0, ans=0;
    for(int i=0;i<nums.size();i++){
        if(nums[i]%2==0) cnt++;
        if(i>=k && nums[i-k]%2==0) cnt--;
        ans=max(ans, cnt);
    }
    return ans;
}

```

6. Max Prime Numbers in Subarray of Size k

Q: Count how many primes appear in subarray of size k, return max.

Trick: Precompute isPrime.

```

bool isPrime(int x){
    if(x<2) return false;
    for(int i=2;i*i<=x;i++) if(x%i==0) return false;
    return true;
}
int maxPrimes(vector<int>& nums, int k) {
    int cnt=0, ans=0;
    for(int i=0;i<nums.size();i++){
        if(isPrime(nums[i])) cnt++;
        if(i>=k && isPrime(nums[i-k])) cnt--;
        ans=max(ans, cnt);
    }
    return ans;
}

```

◆ Patterns You Can Derive

- **Chars:** vowels, consonants, uppercase, lowercase, digits.
- **Numbers:** evens, odds, primes, negatives, positives, multiples of k.

- **Special properties:** distinct count, freq of most occurring char, palindromic chars (if asked).
-

◆ Recognition Trick (Summary)

👉 Whenever you see:

- Substring/subarray of **fixed length k**
- Count **how many elements satisfy a property**
- Asked for **max/min count**

💡 Immediately think: **Fixed-size Frequency Sliding Window.**

Sliding window maximum (deque)

🔗 Core Idea of "Sliding Window Maximum"

- You have a window (usually fixed-size k), and at every position you want the **max (or min) element inside the window.**
 - A **deque** (double-ended queue) helps maintain the "candidates" for max/min in decreasing (or increasing) order.
 - Trick: The **front of the deque is always the answer for the current window.**
-

🌐 How to Recognize It

👉 If the problem asks:

- "Max/Min element in every window of size k "
- "Minimize/maximize something across subarrays of size k "
- "Maintain rolling max/min efficiently (better than $O(n*k)$)"

Then → **Use Monotonic Deque (Sliding Window Maximum pattern).**

Variations & Tricks

1. Classic: Max in sliding window of size k

Q: Given an array, return max in every subarray of size k.

Trick: Maintain deque in decreasing order, pop smaller elements when adding new.

If max => monotonic decreasing

If min => monotonic increasing

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq; // stores indices
    vector<int> ans;
    for (int i = 0; i < nums.size(); i++) {
        // remove out-of-window indices
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        // remove smaller elements (they can never be max if a bigger one
        exists after them)
        while (!dq.empty() && nums[dq.back()] < nums[i]) dq.pop_back();
        dq.push_back(i);
        // record answer
        if (i >= k - 1) ans.push_back(nums[dq.front()]);
    }
    return ans;
}
```

2. Min in sliding window of size k

Same as above but maintain **increasing deque**.

```
vector<int> minSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> ans;
    for (int i = 0; i < nums.size(); i++) {
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        while (!dq.empty() && nums[dq.back()] > nums[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) ans.push_back(nums[dq.front()]);
    }
    return ans;
}
```

3. Sum of max elements in all windows of size k

Q: Find sum of maximum element in each window.

Trick: Same deque as (1), just accumulate instead of pushing into result array.

```
long long sumOfMaxInWindows(vector<int>& nums, int k) {
    deque<int> dq;
```



```

long long sum = 0;
for (int i = 0; i < nums.size(); i++) {
    if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
    while (!dq.empty() && nums[dq.back()] < nums[i]) dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1) sum += nums[dq.front()];
}
return sum;
}

```

4. Sliding window min+max sum (StockSpan / Min-Max Range Problems)

Q: For every window, add (max+min).

Trick: Maintain two deques simultaneously, one for max, one for min.

```

long long sumOfMinMax(vector<int>& nums, int k) {
    deque<int> maxD, minD;
    long long sum = 0;
    for (int i = 0; i < nums.size(); i++) {
        if (!maxD.empty() && maxD.front() <= i - k) maxD.pop_front();
        if (!minD.empty() && minD.front() <= i - k) minD.pop_front();

        while (!maxD.empty() && nums[maxD.back()] < nums[i]) maxD.pop_back();
        while (!minD.empty() && nums[minD.back()] > nums[i]) minD.pop_back();

        maxD.push_back(i);
        minD.push_back(i);

        if (i >= k - 1) sum += nums[maxD.front()] + nums[minD.front()];
    }
    return sum;
}

```

5. Longest subarray where difference (max-min) ≤ limit

Q (LC 1438): Given array and limit, find longest subarray where max-min ≤ limit.

Trick: Two deques: one maintains max, another min. Expand window, shrink when invalid.

```

int longestSubarray(vector<int>& nums, int limit) {
    deque<int> maxD, minD;
    int left = 0, ans = 0;
    for (int right = 0; right < nums.size(); right++) {
        while (!maxD.empty() && nums[maxD.back()] < nums[right])
            maxD.pop_back();
        while (!minD.empty() && nums[minD.back()] > nums[right])
            minD.pop_back();
        maxD.push_back(right);
        minD.push_back(right);

        while (nums[maxD.front()] - nums[minD.front()] > limit) {
            if (maxD.front() == left) maxD.pop_front();
            if (minD.front() == left) minD.pop_front();
            left++;
        }
        ans = max(ans, right - left + 1);
    }
    return ans;
}

```

```

        left++;
    }
    ans = max(ans, right - left + 1);
}
return ans;
}

```

6. Stock Span Problem (variation)

Q: For each day, find number of consecutive previous days with price $\leq$ today's price.

Trick: Monotonic decreasing stack (close cousin of deque).

Summary Trick to Remember

- **Max in every window?** → Monotonic decreasing deque.
- **Min in every window?** → Monotonic increasing deque.
- **Need both?** → Maintain 2 deques.
- **Window length flexible (shrink/expand)?** → Maintain max/min deques and shrink until condition valid.

👉 Mnemonic:

- *Deque Front = Answer*
- *Pop Back = Maintain Order*
- *Pop Front = Out of window*

Problem Statement

Given a string s and an integer k , find the **length of the longest substring** that contains **at least k distinct characters**.

Example 1

```

s = "aabbcc"
k = 2
Output = 4
Explanation: "aabb" or "bbcc" are valid.

```

Example 2

```

s = "abcde"
k = 3
Output = 3
Explanation: "abc", "bcd", "cde" all valid.

```

◆ Pattern

This belongs to the **Sliding Window + HashMap (Two Pointers)** category.

- Similar to “**Longest Substring with At Most K Distinct Characters**”.
 - Only difference: here we need **at least K** instead of “at most K”.
-

◆ Trick to Solve

1. Use **two pointers (left, right)** and a **hashmap** to store frequency of characters.
 2. Expand `right` pointer and insert characters into the hashmap.
 3. When the hashmap has $\geq k$ **distinct characters**, update `maxLen`.
 4. If hashmap size drops below `k` (when moving `left`), it becomes invalid $\rightarrow$ shrink window.
 5. Continue until the end.
-

◆ Variations

- **Longest Substring with At Most K Distinct Characters** $\rightarrow$ Classic problem, easier.
 - **Longest Substring with Exactly K Distinct Characters** $\rightarrow$ Trick:
`len(atMostK) - len(atMost(K-1))`
 - **Smallest substring with at least K distinct characters** $\rightarrow$ Similar sliding window but minimize instead of maximize.
-

◆ Optimized C++ Solution (O(N))

```
#include <bits/stdc++.h>
using namespace std;

int longestSubstringAtLeastKDistinct(string s, int k) {
    int n = s.size();
    unordered_map<char, int> freq;
    int left = 0, maxLen = 0;

    for (int right = 0; right < n; right++) {
        freq[s[right]]++;

        // As soon as we have >= k distinct chars, update answer
        while ((int)freq.size() >= k) {
            maxLen = max(maxLen, right - left + 1);

            // shrink window if still valid
            freq[s[left]]--;
            if (freq[s[left]] == 0)
                left++;
        }
    }

    return maxLen;
}
```

```

        freq.erase(s[left]);
        left++;
    }
}
return maxLen;
}

int main() {
    string s = "aabbcc";
    int k = 2;
    cout << longestSubstringAtLeastKDistinct(s, k) << endl; // Output: 4
    return 0;
}

```

◆ Complexity

- **Time:** $O(N)$ → each char visited at most twice.
 - **Space:** $O(K)$ → hashmap storing distinct chars.
-

✓ Trick to Remember

- For **at least K** → expand until valid, then shrink but still count lengths.
 - For **exactly K** → use $\text{atMost}(K) - \text{atMost}(K-1)$.
 - For **at most K** → direct sliding window.
-

At most K distinct: Longest substring with at most K distinct.

Pattern: At Most K Distinct (Variable Window)

- **Idea:** Keep expanding the window, track frequency of characters (or numbers), shrink the window when distinct count $> K$.
 - **Key Tool:** **HashMap/HashTable** to count frequencies.
 - **Trick:**
 - "Longest with at most K distinct" → Expand until more than K distinct, then shrink.
 - "Longest with exactly K distinct" = $\text{atMost}(K) - \text{atMost}(K-1)$
-



Variations

1. Longest Substring with At Most K Distinct Characters

👉 *Classic base problem*

- Input: "eceba", k = 2
- Output: 3 ("ece")
- **Trick:** Keep `map<char, int>`, shrink when `distinct > k`.

```
int lengthOfLongestSubstringKDistinct(string s, int k) {
    int n = s.size(), left = 0, ans = 0;
    unordered_map<char, int> freq;
    for (int right = 0; right < n; right++) {
        freq[s[right]]++;
        while (freq.size() > k) { //not valid case
            if (--freq[s[left]] == 0) freq.erase(s[left]);
            left++;
        }
        ans = max(ans, right - left + 1);
    }
    return ans;
}
```

2. Longest Substring with Exactly K Distinct Characters

👉 Use trick: `atMost(K) - atMost(K-1)`

- Input: "aaabbc", k = 2
- Output: 5 ("aaabb")

```
int atMostK(string &s, int k) {
    int n = s.size(), left = 0, ans = 0;
    unordered_map<char, int> freq;
    for (int right = 0; right < n; right++) {
        freq[s[right]]++;
        while (freq.size() > k) {
            if (--freq[s[left]] == 0) freq.erase(s[left]);
            left++;
        }
        ans += right - left + 1; // count all substrings
    }
    return ans;
}

int lengthOfLongestSubstringKDistinct(string s, int k) {
    return atMostK(s, k) - atMostK(s, k - 1);
}
```

```
}
```

3. Fruit Into Baskets (Leetcode 904)

👉 *Longest subarray with at most 2 distinct numbers*

- Input: [1,2,1,2,3]
- Output: 4 ([1,2,1,2])
- Trick: Just set k=2.

```
int totalFruit(vector<int>& fruits) {
    int n = fruits.size(), left = 0, ans = 0;
    unordered_map<int,int> freq;
    for (int right = 0; right < n; right++) {
        freq[fruits[right]]++;
        while (freq.size() > 2) {
            if (--freq[fruits[left]] == 0) freq.erase(fruits[left]);
            left++;
        }
        ans = max(ans, right - left + 1);
    }
    return ans;
}
```

4. Longest Substring Without Repeating Characters

👉 Special case of atMost K where **K = infinite but no duplicates allowed.**

- Input: "abcabcbb"
- Output: 3 ("abc")

```
int lengthOfLongestSubstring(string s) {
    unordered_map<char,int> freq;
    int left = 0, ans = 0;
    for (int right = 0; right < s.size(); right++) {
        freq[s[right]]++;
        while (freq[s[right]] > 1) { // no duplicates allowed
            if (--freq[s[left]] == 0) freq.erase(s[left]);
            left++;
        }
        ans = max(ans, right - left + 1);
    }
    return ans;
}
```

5. Longest Substring with At Most K Repeating Characters (*variation*)

👉 E.g., *Leetcode 395 / 424 style*

- Keep window, but shrink when frequency rules break.
- Example: Longest substring with same letter after k replacements (we already discussed earlier).

6. Subarrays with K Different Integers (Leetcode 992)

👉 Count **number of subarrays**, not just max length.

- Trick: Use formula $\text{atMost}(k) - \text{atMost}(k-1)$.
- Input: $[1, 2, 1, 2, 3]$, $k = 2$
- Output: 7

```
int atMost(vector<int>& nums, int k) {
    unordered_map<int,int> freq;
    int left = 0, count = 0;
    for (int right = 0; right < nums.size(); right++) {
        freq[nums[right]]++;
        while (freq.size() > k) {
            if (--freq[nums[left]] == 0) freq.erase(nums[left]);
            left++;
        }
        count += right - left + 1; // count subarrays
    }
    return count;
}

int subarraysWithKDistinct(vector<int>& nums, int k) {
    return atMost(nums, k) - atMost(nums, k - 1);
}
```

Quick Tricks to Remember

- **At Most K Distinct (length)** → Expand, shrink when $\text{freq.size()} > K$, take max length.
- **Exactly K Distinct** → $\text{atMost}(K) - \text{atMost}(K-1)$
- **Count Subarrays** → same trick, but add $(\text{right}-\text{left}+1)$ each step.
- **Fruit Basket / Longest 2 types** → Special case of $K=2$.
- **No Repeating Substring** → Special case of $\text{atMost}(K=\infty)$ but forbid duplicates.

⚡ So summary:

- All problems in this family = “**shrink when constraint breaks**”.
- Variation = *Max length* OR *Count of subarrays* OR *Exactly K distinct*.

K = 2 specialization: Fruit into baskets

 Core Idea: At Most K Distinct → Specializing to K=2

- Maintain a sliding window $[l, r]$ with a frequency map.
- Expand r to include new fruits.
- Shrink l if the map has more than **2 distinct** fruits.
- The window size = number of fruits picked.

Trick to Remember:

"Fruit into baskets = longest subarray with at most **2 distinct elements**."

So just remember → **K=2 = Two baskets**.

Variations of This Pattern (K=2 specialization)

1. Fruit Into Baskets (LeetCode 904)

You have a row of trees, each with a fruit type. Pick from two baskets max.

Goal: Longest subarray with at most 2 distinct values.

C++ Code:

```
int totalFruit(vector<int>& fruits) {
    unordered_map<int,int> freq;
    int l = 0, ans = 0;
    for(int r = 0; r < fruits.size(); r++){
        freq[fruits[r]]++;
        while(freq.size() > 2){
            if(--freq[fruits[l]] == 0) freq.erase(fruits[l]);
            l++;
        }
        ans = max(ans, r-l+1);
    }
    return ans;
}
```

2. Longest Substring with At Most Two Distinct Characters (LeetCode 159)

Given a string, return length of longest substring containing at most 2 distinct characters.

C++ Code:

```
int lengthOfLongestSubstringTwoDistinct(string s) {
    unordered_map<char,int> freq;
    int l = 0, ans = 0;
    for(int r = 0; r < s.size(); r++){
```



```

        freq[s[r]]++;
        while(freq.size() > 2){
            if(--freq[s[l]] == 0) freq.erase(s[l]);
            l++;
        }
        ans = max(ans, r-l+1);
    }
    return ans;
}

```

👉 Trick: **Fruit baskets but with characters instead of fruits.**

3. Longest Subarray with At Most Two 0s / 1s

(Sometimes hidden in interview variations)

Example: Binary array, find longest subarray with at most 2 zeros.

(This is **Fruit into baskets** where the “fruits” are 0/1, and K=2 means both 0 and 1 allowed).

✅ Code: (same logic)

```

int longestWithAtMostTwoZeros(vector<int>& nums) {
    unordered_map<int,int> freq;
    int l=0, ans=0;
    for(int r=0; r<nums.size(); r++){
        freq[nums[r]]++;
        while(freq.size() > 2){
            if(--freq[nums[l]]==0) freq.erase(nums[l]);
            l++;
        }
        ans = max(ans, r-l+1);
    }
    return ans;
}

```

👉 Trick: If input domain is small (binary array), then $\text{freq.size()} \leq 2$ is always satisfied → answer is just entire array. But if constraint is "at most 2 zeros", you just adapt by counting zeros.

4. Longest Substring with At Most Two Distinct Vowels (custom variation)

Example: Given string, find longest substring with at most 2 distinct vowels.

(Same template, just condition check modified to vowels set).

General Trick to Remember This Pattern:

- Whenever you see “**at most 2 types / characters / values**”, immediately think → **Fruit Into Baskets template**.
- Just plug in your element type (integer / char / fruit).
- Core template:
 - Add `s[r] → freq++`
 - If `freq.size() > 2 → shrink from left`
 - Track `maxLen`

 Mnemonic:

Two baskets = K=2 = longest window with at most 2 distinct.

So all variations:

1. Fruits into baskets (array of ints, at most 2 types).
2. Longest substring with at most 2 distinct (string version).
3. Binary / custom array with at most 2 values.
4. Customized conditions (like vowels, special chars)

Replacement: Longest repeating character replacement

Core Idea: Replacement Pattern

You are allowed to **replace (or modify) some characters/numbers** to make the window valid.

 The trick is:

- Maintain the **frequency of elements in the window**.
- Keep track of the **most frequent element count** (`maxCount`).
- If `(window_size - maxCount) > k → window is invalid → shrink`.
- Otherwise → window is valid → update result.

Why this works:

- To make the window valid, we must replace all non-majority elements.
 - Number of replacements needed = `window_size - maxCount`.
-

Variations + Codes (C++)

1. Longest Repeating Character Replacement

 Q: Replace at most k characters to get the longest substring with repeating characters.

```
int characterReplacement(string s, int k) {
    vector<int> freq(26,0);
    int left=0, maxCount=0, res=0;

    for(int right=0; right<s.size(); right++){
        freq[s[right]-'A']++;
        maxCount = max(maxCount, freq[s[right]-'A']);

        while((right-left+1) - maxCount > k){
            freq[s[left]-'A']--;
            left++;
        }
        res = max(res, right-left+1);
    }
    return res;
}
```

 Trick: **Window length – max freq $\leq k$**

2. Longest Subarray of 1s After Replacement

 Q: Given a binary array, replace at most k zeros with ones. Find longest subarray of 1s.

```
int longestOnes(vector<int>& nums, int k) {
    int left=0, zeroCount=0, res=0;

    for(int right=0; right<nums.size(); right++){
        if(nums[right]==0) zeroCount++;

        while(zeroCount > k){
            if(nums[left]==0) zeroCount--;
            left++;
        }
        res = max(res, right-left+1);
    }
    return res;
}
```

 Trick: Track count of **zeros** directly instead of frequency.

3. Longest Substring with Same Letters After Replacement (Generalized)

👉 Q: Similar to #1, but with lowercase letters or unicode.

Same logic, just expand frequency array (26 → 128).

4. Max Consecutive Answers (Leetcode 2024)

👉 Q: Exam answers are "T"/"F". You can change at most k answers to make all answers in substring the same.

```
int maxConsecutiveAnswers(string s, int k) {
    auto helper = [&](char target){
        int left=0, count=0, res=0;
        for(int right=0; right<s.size(); right++){
            if(s[right]!=target) count++;
            while(count > k){
                if(s[left]!=target) count--;
                left++;
            }
            res = max(res, right-left+1);
        }
        return res;
    };
    return max(helper('T'), helper('F'));
}
```

🔑 Trick: Run **two passes** – one treating "T" as majority, one treating "F".

5. Longest Subarray After At Most K Operations (Generic)

👉 Q: Array of integers, you can increment/decrement at most k numbers to make them equal. Max subarray length?

(Similar to #1, maintain counts and majority element.)

🔗 Tricks to Remember This Pattern:

1. **Check "replacement" keyword:** Replace/change/flip/convert → it's this pattern.
2. **Formula:** `window_size - max_freq <= k`.
3. **For binary arrays:** Just track `zeroCount` (simpler).
4. **For yes/no answers:** Try both options separately.

⚡ Quick Table for Recall:

Variation	What to Replace?	Key Trick
Longest Repeating Char Replacement	Any char	$\text{win} - \text{maxFreq} \leq k$
Longest 1s after Flip	$0 \rightarrow 1$	Track $\text{zeroCount} \leq k$
Max Consecutive Answers	T/F	Check both with helper
Generic Replace	Any element	Frequency + majority

◆ Pattern 1: Zeros Flips (Max Consecutive Ones III)

Base Problem

- **Q:** Given a binary array *nums* and an integer *k*, return the length of the longest subarray containing at most *k* zeros.
- **Trick to remember:** 🖱️ "Allow K mistakes (zeros), maximize consecutive 1s."

✓ Variations of Zeros Flips

1. Max Consecutive Ones III (LC 1004)

```
2. int longestOnes(vector<int>& nums, int k) {
3.     int l = 0, zeros = 0, ans = 0;
4.     for (int r = 0; r < nums.size(); r++) {
5.         if (nums[r] == 0) zeros++;
6.         while (zeros > k) {
7.             if (nums[l] == 0) zeros--;
8.             l++;
9.         }
10.        ans = max(ans, r - l + 1);
11.    }
12.    return ans;
13. }
```

- Allow at most *k* zeros.

2. Longest Subarray After Deleting One (LC 1493)

- Similar but you must **delete exactly one element**, so allow $k = 1$ zero but subtract 1 at the end.

```

3. int longestSubarray(vector<int>& nums) {
4.     int l = 0, zeros = 0, ans = 0;
5.     for (int r = 0; r < nums.size(); r++) {
6.         if (nums[r] == 0) zeros++;
7.         while (zeros > 1) {
8.             if (nums[l] == 0) zeros--;
9.             l++;
10.        }
11.        ans = max(ans, r - l); // delete 1 element
12.    }
13.    return ans;
14. }

```

3. Longest Substring with At Most K Replacements (generalized)

- Instead of only flipping zeros → ones, allow any character replacement.
 - Closest variant is LC 424. **Longest Repeating Character Replacement** (we already covered earlier, but it fits this category too).
-

👉 Trick to Recognize:

- If you see “*flip at most k things / delete at most k elements / tolerate k errors*” → It’s **Zeros Flips Pattern**.
-

◆ Pattern 2: Sum Threshold (Min Subarray Length $\geq$ Target)

Base Problem

- **Q:** Given array *nums* of positive integers and *target*, return the minimal length of a subarray with $\text{sum} \geq \text{target}$.
 - **Trick to remember:** 👉 "Grow window till valid, then shrink to minimum."
-

✓ Variations of Sum Threshold

1. LC 209. Minimum Size Subarray Sum

```

2. int minSubArrayLen(int target, vector<int>& nums) {
3.     int l = 0, sum = 0, ans = INT_MAX;

```

```

4.     for (int r = 0; r < nums.size(); r++) {
5.         sum += nums[r];
6.         while (sum >= target) {
7.             ans = min(ans, r - l + 1);
8.             sum -= nums[l++];
9.         }
10.    }
11.    return ans == INT_MAX ? 0 : ans;
12. }

```

2. Smallest Subarray with Sum Greater than X (GFG)

- Same as above, but phrased differently ($\geq$ vs $>$).
-

3. Longest Subarray with Sum $\leq K$ (variation)

- Instead of min length $\geq$ target, maximize subarray length $\leq K$.
- Trick: Works if nums are positive (shrinking window guarantees monotonicity).

```

4. int longestSubarraySumLEK(vector<int>& nums, int k) {
5.     int l = 0, sum = 0, ans = 0;
6.     for (int r = 0; r < nums.size(); r++) {
7.         sum += nums[r];
8.         while (sum > k) sum -= nums[l++];
9.         ans = max(ans, r - l + 1);
10.    }
11.    return ans;
12. }

```

👉 Trick to Recognize:

- If you see “*minimum/maximum length subarray with sum constraint ($\geq$ $\leq$)*” → It’s **Sum Threshold Pattern**.
-

◆ Summary (Top 3 Variations Each)

Zeros Flips

- Max Consecutive Ones III** → allow k zeros.
- Longest Subarray After Deleting One** → allow 1 zero, subtract 1.
- Longest Repeating Character Replacement** → allow k replacements (generalized).

Sum Threshold

- Min Subarray Length $\geq$ Target** → shrink to minimum.

2. **Smallest Subarray Sum $> X$** $\rightarrow$ same concept, slight variation.
3. **Longest Subarray with Sum $\leq K$** $\rightarrow$ maximize length under cap.

Pattern 7: Product threshold (Subarray product $< k$)

 When the condition involves the **product** of elements instead of sum/frequency.

Core Problem

LeetCode 713: Subarray Product Less Than K

Count subarrays where product $< k$.

- Trick: Just like sum $\geq k$ problems, but instead of subtracting, you **divide** when product $\geq k$.
- Sliding window with multiplication/division.

```
int numSubarrayProductLessThanK(vector<int>& nums, int k) {
    if(k <= 1) return 0;
    int n = nums.size(), left = 0;
    long long prod = 1, ans = 0;
    for(int right = 0; right < n; right++) {
        prod *= nums[right];
        while(prod >= k) prod /= nums[left++];
        ans += right - left + 1; // all subarrays ending at right
    }
    return ans;
}
```

Top 3 Variations

1. Longest Subarray with Product $< k$

- Instead of *counting*, just track the **max window length**.
- Trick: When product $\geq k \rightarrow$ shrink window.

```
2. int longestSubarrayProductLessThanK(vector<int>& nums, int k) {
3.     if(k <= 1) return 0;
4.     int left = 0, ans = 0;
5.     long long prod = 1;
6.     for(int right = 0; right < nums.size(); right++) {
7.         prod *= nums[right];
8.         while(prod >= k) prod /= nums[left++];
9.         ans = max(ans, right - left + 1);
}
```



```

10.     }
11.     return ans;
12. }

```

13. Min Subarray Length with Product $\geq k$

- Opposite version: find the shortest subarray such that product $\geq k$.
- Trick: Just like "min sum $\geq$ target", but product instead of sum.

```

14. int minSubarrayProductAtLeastK(vector<int>& nums, int k) {
15.     int left = 0, n = nums.size(), ans = INT_MAX;
16.     long long prod = 1;
17.     for(int right = 0; right < n; right++) {
18.         prod *= nums[right];
19.         while(prod >= k) {
20.             ans = min(ans, right - left + 1);
21.             prod /= nums[left++];
22.         }
23.     }
24.     return ans == INT_MAX ? 0 : ans;
25. }

```

26. Count Subarrays with Product in [L, R]

- Trick: Reduce to two "less than" problems:
count(prod $\leq$ R) - count(prod < L)

```

27. int countSubarrayProductRange(vector<int>& nums, int L, int R) {
28.     auto countLess = [&](int k) {
29.         if(k <= 1) return 0;
30.         int left = 0, res = 0;
31.         long long prod = 1;
32.         for(int right = 0; right < nums.size(); right++) {
33.             prod *= nums[right];
34.             while(prod >= k) prod /= nums[left++];
35.             res += right - left + 1;
36.         }
37.         return res;
38.     };
39.     return countLess(R+1) - countLess(L);
40. }

```

Trick to Remember

- For **product problems**, always:
 - multiply $\rightarrow$ check $\rightarrow$ divide.
 - Counting? Add (right - left + 1).
 - Length? Just take max.
 - Min length? Shrink as much as possible.

Pattern 8: Covering (Minimum Window Substring)

👉 When the problem asks for **covering all required elements/characters** in the smallest/largest window.

✓ Core Problem

LeetCode 76: Minimum Window Substring

Find the smallest substring of *s* that contains all characters of *t*.

- Trick: Maintain *need* map (char → count).
- Keep shrinking window when all requirements are met.

```
string minWindow(string s, string t) {
    unordered_map<char,int> need;
    for(char c : t) need[c]++;
    int required = need.size();
    int formed = 0;
    unordered_map<char,int> window;
    int l = 0, r = 0, minLen = INT_MAX, start = 0;

    while(r < s.size()) {
        char c = s[r++];
        window[c]++;
        if(need.count(c) && window[c] == need[c]) formed++;

        while(l <= r && formed == required) {
            if(r - l < minLen) minLen = r - l, start = l;
            char d = s[l++];
            window[d]--;
            if(need.count(d) && window[d] < need[d]) formed--;
        }
    }
    return minLen == INT_MAX ? "" : s.substr(start, minLen);
}
```

🔑 Top 3 Variations

1. Check if Permutation Exists (LeetCode 567)

Does *s1*'s permutation exist in *s2*?

- Trick: Fixed window size = *s1.length()*. Just compare counts.

```
2. bool checkInclusion(string s1, string s2) {
3.     if(s1.size() > s2.size()) return false;
4.     vector<int> cnt1(26,0), cnt2(26,0);
5.     for(char c : s1) cnt1[c-'a']++;
6.     for(int i=0; i<s1.size(); i++) cnt2[s2[i]-'a']++;
7.     if(cnt1 == cnt2) return true;
8.     for(int i=s1.size(); i<s2.size(); i++) {
9.         cnt2[s2[i]-'a']++;
10.        cnt2[s2[i-s1.size()]-'a']--;
11.        if(cnt1 == cnt2) return true;
    }
```

```

12.     }
13.     return false;
14. }

```

15. Smallest Window Containing All Characters of Another String (LeetCode 727)

Subsequence instead of substring.

- Trick: Two pointers, greedy match.
- Variation of covering but slower ($O(n \cdot m)$).

16. Substring with All Characters of the Alphabet (custom variation)

Find shortest substring containing all 26 letters.

- Trick: Exactly same as **minimum window substring**, but need has 26 letters with count 1.

Trick to Remember

- Covering → "**Do I have all ingredients?**"
- Use **need map** and **formed counter**.
- While all covered → shrink window.
- If checking permutation → fixed window size.

✓ So summary:

- **Product problems** = multiply/divide window → count or length.
- **Covering problems** = need map + formed count → expand + shrink.

Pattern 1: Anagrams (fixed-size window + counts)

👉 Core idea: Maintain a frequency map for target & sliding window of same size → compare.

👉 Trick: Think of it as “**permutation match inside a sliding window of fixed size**”.

✓ Variation 1: Find All Anagrams in a String

- **Q:** Given strings s and p , return all start indices of p 's anagrams in s .
- **Trick:** Window size = $\text{len}(p)$. Keep frequency maps and compare.

```

vector<int> findAnagrams(string s, string p) {
    vector<int> res;
    if (s.size() < p.size()) return res;

```

```

vector<int> cnt(26, 0), window(26, 0);
for (char c : p) cnt[c - 'a']++;

int k = p.size();
for (int i = 0; i < s.size(); i++) {
    window[s[i] - 'a']++;
    if (i >= k) window[s[i-k] - 'a']--;
    if (window == cnt) res.push_back(i - k + 1);
}
return res;
}

```

✓ Variation 2: Permutation in String

- **Q:** Return true if s2 contains any permutation of s1.
- **Trick:** Same as above but stop when found.

```

bool checkInclusion(string s1, string s2) {
    if (s1.size() > s2.size()) return false;

    vector<int> cnt(26, 0), window(26, 0);
    for (char c : s1) cnt[c - 'a']++;

    int k = s1.size();
    for (int i = 0; i < s2.size(); i++) {
        window[s2[i] - 'a']++;
        if (i >= k) window[s2[i-k] - 'a']--;
        if (window == cnt) return true;
    }
    return false;
}

```

✓ Variation 3: Count Anagram Substrings

- **Q:** Count how many substrings of s are anagrams of p.
- **Trick:** Same as findAllAnagrams but instead of storing indices, count them.

```

int countAnagrams(string s, string p) {
    if (s.size() < p.size()) return 0;
    vector<int> cnt(26, 0), window(26, 0);
    for (char c : p) cnt[c - 'a']++;

    int k = p.size(), ans = 0;
    for (int i = 0; i < s.size(); i++) {
        window[s[i] - 'a']++;
        if (i >= k) window[s[i-k] - 'a']--;
        if (window == cnt) ans++;
    }
    return ans;
}

```



Trick to remember Anagrams pattern:

“Fixed size + frequency count match” → Whenever the problem says **anagram** / **permutation**, think *frequency vector* + *fixed window*.

◆ Pattern 2: Exactly K distinct (counting)



Core idea: Use `atMost(K)` helper, then
`exactlyK = atMost(K) - atMost(K-1)`



Trick: Think of it as “count all subarrays with $\leq K$ distinct, subtract $\leq K-1$ ”.



Variation 1: Subarrays with K Different Integers

- **Q:** Count subarrays with exactly K distinct integers.
- **Trick:** At most function → prefix trick.

```
int atMostK(vector<int>& nums, int K) {
    unordered_map<int,int> freq;
    int i = 0, ans = 0;
    for (int j = 0; j < nums.size(); j++) {
        if (freq[nums[j]]++ == 0) K--;
        while (K < 0) {
            if (--freq[nums[i]] == 0) K++;
            i++;
        }
        ans += j - i + 1;
    }
    return ans;
}

int subarraysWithKDistinct(vector<int>& nums, int K) {
    return atMostK(nums, K) - atMostK(nums, K - 1);
}
```

```
class Solution {
public:
    int atMost(vector<int> &nums, int k){
        unordered_map<int,int> freq;
        int ans = 0;
        int l = 0;
        for(int r=0; r<nums.size(); r++){
            freq[nums[r]]++;

            while(freq.size() > k){
                if(--freq[nums[l]] == 0) freq.erase(nums[l]);
                l++;
            }
            ans += r - l + 1;
        }
        return ans;
    }
};
```

```

        }
        ans += r-l+1;
    }
    return ans;
}
int subarraysWithKDistinct(vector<int>& nums, int k) {

    return atMost(nums,k) - atMost(nums,k-1);
}
};

```

✓ Variation 2: Longest Substring with Exactly K Distinct Characters

- **Q:** Given string s , find length of longest substring with exactly K distinct chars.
- **Trick:** Same `atMost` trick but store max instead of count.

```

int longestSubstringKDistinct(string s, int K) {
    unordered_map<char,int> freq;
    int i = 0, ans = 0, distinct = 0;
    for (int j = 0; j < s.size(); j++) {
        if (freq[s[j]]++ == 0) distinct++;
        while (distinct > K) {
            if (--freq[s[i]] == 0) distinct--;
            i++;
        }
        if (distinct == K) ans = max(ans, j - i + 1);
    }
    return ans;
}

```

✓ Variation 3: Count Substrings with Exactly K Distinct Characters

- **Q:** Count substrings of string s with exactly K distinct chars.
- **Trick:** Same as array version $\rightarrow$ `atMost(K) - atMost(K-1)`.

```

int atMostKDistinct(string s, int K) {
    unordered_map<char,int> freq;
    int i = 0, ans = 0;
    for (int j = 0; j < s.size(); j++) {
        if (freq[s[j]]++ == 0) K--;
        while (K < 0) {
            if (--freq[s[i]] == 0) K++;
            i++;
        }
        ans += j - i + 1;
    }
    return ans;
}

```

```
int substringsWithKDistinct(string s, int K) {  
    return atMostKDistinct(s, K) - atMostKDistinct(s, K - 1);  
}
```

📌 **Trick to remember Exactly K pattern:**

Whenever you see “**exactly K distinct**” → think

➡ `atMost(K) - atMost(K-1)`

This is the key shortcut.

✓ Now you have **both families**:

- **Anagrams** → fixed-size + freq vector
- **Exactly K** → `atMost(K) - atMost(K-1)`